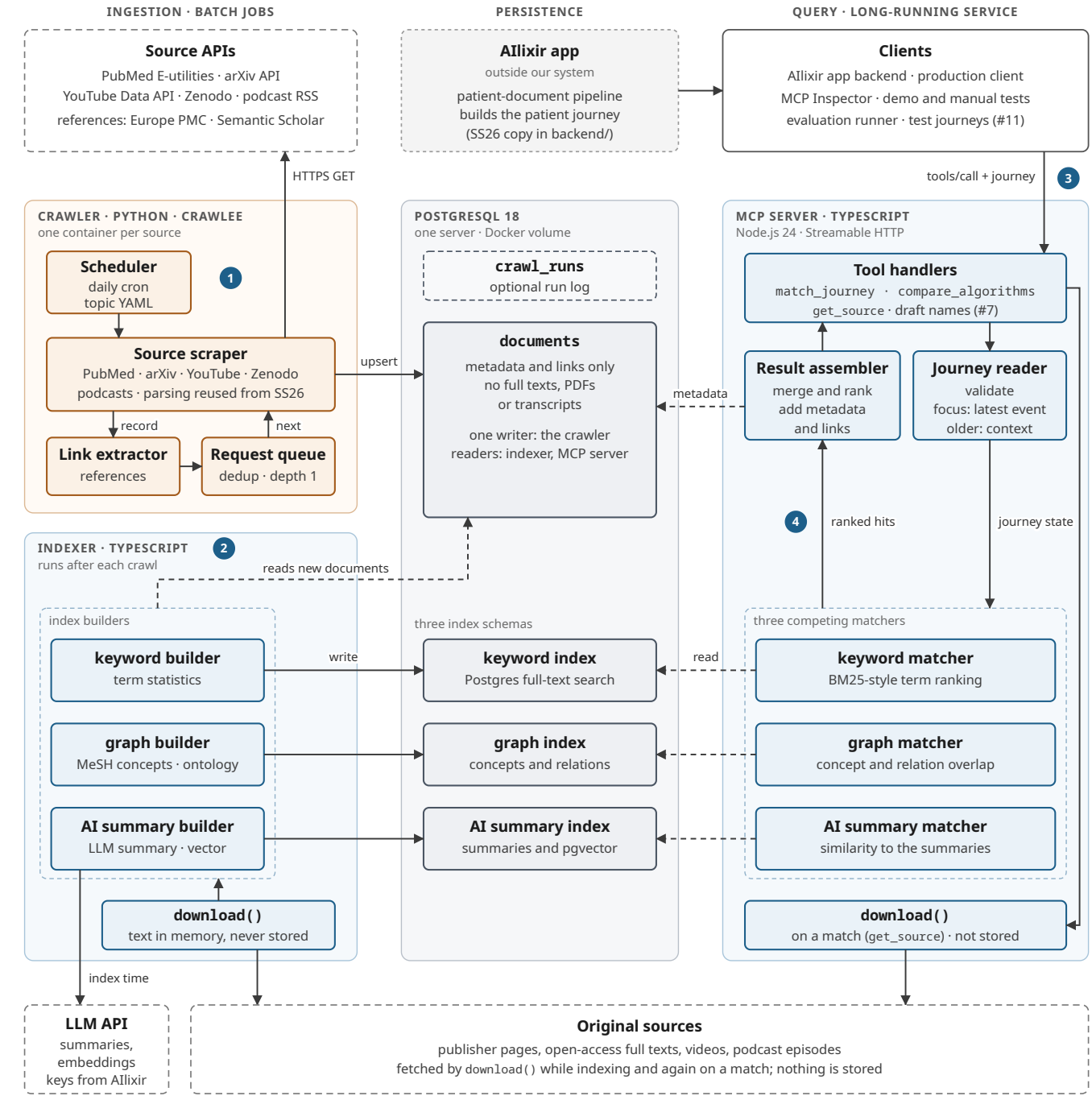
1 Runtime components (tiers)

What runs and who calls whom. Ingestion runs as scheduled batch jobs, all data is in one PostgreSQL server, and clients talk to one long-running MCP server. The numbered steps are explained on page 4.



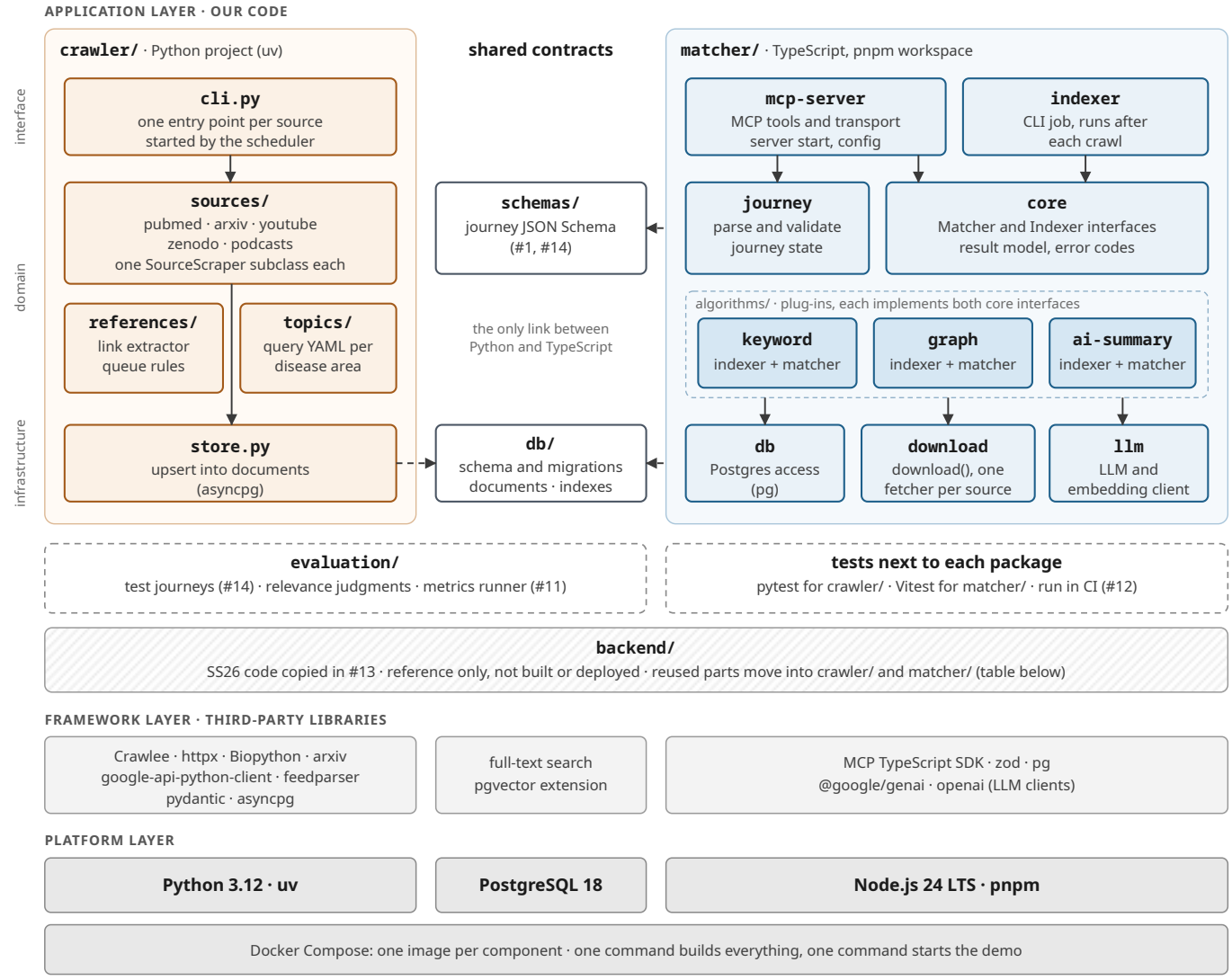
Python component · TypeScript component · PostgreSQL table or schema · external service · outside our system

call or write ------ read

- Every lane runs as containers from one Docker Compose file: PostgreSQL, one crawler container per source, the indexer job and the MCP server.
- Each table has one writer. The crawler writes `documents`, the indexer writes the three indexes, and the MCP server only reads.
- `download()` is one TypeScript module used by the indexer and the MCP server. Full texts stay in memory and are never written to the database.

2 Code components (layers)

How the code in this repository is organised. Each layer only uses the layers below it. Inside our code, interface modules call domain modules, and domain modules call infrastructure modules.



Python module · TypeScript package · algorithm plug-in · shared contract file · dev only, not shipped
reference code · imports · follows the contract

Where the SS26 code in `backend/` goes

SS26 code (`backend/…`)	Target	How
`scrapers/…/PubMed` , `Archive` (arXiv), `YouTube` (official API part)	`crawler/sources/`	Reuse the search and parsing code. Requests go through Crawlee. PDF downloads, chunking and the AstraDB embedding are dropped.
`scrapers/…/BaseScraper` (stable IDs, dedup)	`crawler/sources/` base class	Adapt: split fetching and parsing from indexing.
`scrapers/key_words/*.csv` , `scrapers/topics.py`	`crawler/topics/`	Reuse as seed queries, converted to one YAML file per disease area.
`workers/pipeline/graph/medical_schema.py`	`matcher/algorithms/graph`	Port the entity and relation types to TypeScript.
`api/chat_pipeline/paper_retriever.py` , `availability.py`	`matcher/core` , `download`	Port the result shape (passages with source references), timeouts and the circuit breaker.
`workers/connections/gemini_client.py`	`matcher/llm`	Port the pacing and retry conventions.
`api/` , `workers/` document pipeline, `shared/` (Firestore, GCS, Pub/Sub)	not in the first release	Stays on the AIlixir side. Reference for a later adapter that builds journeys from patient documents.

Folder names are the planned layout. Today the repository holds `backend/` , `Documentation/` and `Deliverables/` .

3 Technology stack

The first-level technologies we build on, grouped by layer. Versions are the current releases on 27.10.2026. The lockfiles pin exact versions, and the bill of materials in the planning document (#16) is the binding list. Course rule: no copyleft licences in the product.

Technology	Version	Used for	Licence	Basis
Languages and runtimes				
Python	3.12	Crawler	PSF-2.0	Partner
TypeScript on Node.js	Node 24 LTS	Indexer, matchers, MCP server	Apache-2.0 / MIT	Partner
Crawling (Python)				
Crawlee for Python	1.11	Request queue with dedup, retries and rate limits	Apache-2.0	#5
Biopython (Entrez)	1.88	PubMed search and record parsing	Biopython (BSD-style)	#13
arxiv	4.0	arXiv API client	MIT	#13
google-api-python-client	2.201	YouTube Data API, official API only	Apache-2.0	#5
feedparser	6.0	Podcast RSS feeds	BSD-2-Clause	Proposal
pydantic	2.14	Validating source records	MIT	#13
asyncpg	0.32	Writing to `documents`	Apache-2.0	Proposal
Storage				
PostgreSQL	18	`documents`, run log, three index schemas	PostgreSQL	#5 Proposal
Postgres full-text search	built in	Keyword index: term statistics and ranking	PostgreSQL	Proposal
pgvector	0.8	Vectors for the AI summary index	PostgreSQL	Proposal
Indexing, matching and MCP interface (TypeScript)				
MCP TypeScript SDK	1.32	MCP server over Streamable HTTP	MIT, moving to Apache-2.0	Partner
zod	4.6	Tool schemas and journey validation	MIT	Proposal
pg (node-postgres)	8.23	Database access	MIT	Proposal
AI services (hosted, keys from AIlixir)				
Gemini via Vertex AI (@google/genai)	SDK 2.31	Source summaries at index time	Apache-2.0 (SDK)	#13
OpenAI embeddings (openai)	SDK 7.45	Vectors for the AI summary index	Apache-2.0 (SDK)	#13
Build, test and delivery				
Docker Compose	5.6	One command to build, one to start the demo	Apache-2.0	Proposal
uv · pnpm	0.13 · 12.15	Dependencies and lockfiles for the bill of materials	MIT / Apache-2.0 · MIT	Proposal
pytest · Vitest · Ruff · Biome	9.1 · 5.0 · 0.17 · 2.6	Unit tests, linting and formatting	MIT; Biome also Apache-2.0	Proposal
GitHub Actions	service	Tests and lint on every pull request	service terms	Proposal
Demo and test data				
MCP Inspector	2.12	Demo client for reviews	MIT / Apache-2.0	Proposal
meddocgenerator (AIlixir)	main	Synthetic patient journeys	Apache-2.0, headers to clarify	Partner

Basis: Partner project description or AIlixir · #5 crawler evaluation · #13 reuse plan · Proposal this document, to be confirmed in review.

Not used, and why

- **JValue crawler:** built for bulk files in FTP folder trees, supports none of our sources, needs RabbitMQ, MinIO and Kubernetes (#5).
- **AstraDB, Vertex AI Ranking:** cloud services, but queries must run against a local database (dropped in #5).
- **Neo4j, Graphiti:** the Neo4j server is GPL-3.0. The patient graph stays with AIlixir; our graph index uses Postgres tables.
- **psycopg:** LGPL-3.0, so we use asyncpg.
- **paperscraper, pypdf, YouTube caption scraping:** we store metadata only, and the caption scraping breaks YouTube's terms.
- **Firebase, Firestore, Cloud Storage, Pub/Sub:** AIlixir app infrastructure, only needed if we take over document upload.

4 Explanation of the diagrams and choices

The Patient Journey Matcher is an MCP server for the AIlixir app. A client sends a patient journey, a dated list of events such as diagnoses and operations, and gets back papers, preprints, videos and podcast episodes that fit the most recent point in that journey.

Runtime view (page 1)

1. **Crawling.** A daily cron job starts one crawler container per source with the queries from a topic file. The crawler searches the source API, follows a paper's references one level deep, and upserts the metadata into `documents`. It stores metadata and links only, never full texts.
2. **Indexing.** After each crawl, the indexer reads the new documents, fetches each text with `download()`, keeps it in memory, and writes three separate indexes: keyword, graph and AI summary. Only the AI summary builder calls an LLM, and only with public source text.
3. **Query.** A client calls an MCP tool with the journey. The journey reader validates it and builds one journey state: the latest event is the focus, and older events are context. How much older events count is not fixed yet; we will test it with the evaluation runner (#11). All three matchers get the same state, so their results can be compared fairly.
4. **Answer.** The result assembler merges and ranks the hits and adds the metadata and links from `documents`. When the client asks for one source, `download()` fetches the original again and passes it on without storing it.

Code view (page 2)

The layers follow the lecture: platform at the bottom, third-party frameworks on top of it, our application code on top of those. The crawler (Python) and the matcher (TypeScript) share no code. Their only contracts are the database schema in `db/` and the journey JSON Schema in `schemas/`, so both sides can be built and tested on their own. Each algorithm is a plug-in that implements the `Indexer` and `Matcher` interfaces from `core`; a fourth approach would not change the server. `backend/` holds the SS26 code copied in #13. It is not built, and the parts we reuse move into the new modules as listed on page 2.

Choices and reasons

- **Python for crawling, TypeScript for indexing and querying.** Required by the project description. The `documents` table is the interface between the two.
- **Our own crawler on Crawlee instead of a fork of the JValue crawler.** This departs from the project description. Our evaluation (#5) found that JValue walks FTP folder trees and stores whole files, while our sources are search APIs that return single records. The professorship advised us to use JValue only as an architecture reference. Crawlee gives us a deduplicating request queue, retries and rate limits, which is what following references needs.
- **Metadata only, download on a match.** Storing the originals is too expensive and often not allowed by their licence. The indexes hold derived data only (term statistics, concepts, summaries), and the original is fetched again when someone needs it.
- **PostgreSQL for documents and indexes.** #5 chose Postgres for `documents`. Full-text search and pgvector cover the keyword and AI indexes in the same server, and the graph index fits in two tables for concepts and relations. One database server keeps the demo to one command, and avoids the GPL-3.0 Neo4j server.
- **Three competing algorithms behind one interface.** AIlixir wants a word-statistics, a graph-based and an AI-based approach, each with its own index. We start with the keyword baseline (#8), then add the graph (#10) and AI summary (#9) approaches, and compare all three on the same test journeys and criteria (#11).
- **Selective reuse of the SS26 code (#13).** We keep the PubMed, arXiv and YouTube parsing, the topic lists and the medical ontology, and port the retrieval result shape and the LLM client conventions to TypeScript. We drop chat, voice, OCR, YouTube caption scraping and the AstraDB index.
- **Health data.** The server does not store or log the journey. Development uses only synthetic journeys from the AIlixir generator (#14).
- **Docker Compose.** One command builds everything and one starts the demo, on any team member's machine, as the build process review on 04.11. requires.

Open points, to settle with AIlixir and in the team

- The journey format the real app sends, and the final MCP tool names and fields (#1, #7).
- MCP transport and authentication between the AIlixir backend and our server.
- LLM provider, EU hosting and who pays; whether summaries and abstracts may be shown to patients.
- Scope of YouTube (stored data must be refreshed or deleted after 30 days) and reference depth beyond one level.
- Production and test system for releases.

Sources: project description, crawler evaluation (#5), reuse plan (#13), issues #1 to #16.